ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN HỆ ĐIỀU HÀNH

BÁO CÁO THỰC HÀNH

MÔN HỆ ĐIỀU HÀNH

Lab 01

Xv6 User Programs: File System

Lớp học phần: Hệ điều hành - CQ2024/22

Sinh viên: 24120438 – Trần Nguyên Tân

24120023 – Nguyễn Thành Bảo

24120040 – Dương Hoài Đức

Giảng viên: ThS. Lê Viết Long

Mục lục

Tóm tắt báo cáo **3**

1 Thông tin thành viên và phân công công việc **4**

2 Thiết lập môi trường trên Windows và macOS **4**

2.1 Windows: cài đặt Ubuntu và các dependencies trên WSL 2 4

2.2 macOS: cài đặt công cụ RISC-V qua Homebrew 5

2.3 Chuẩn bị mã nguồn và khởi chạy xv6 6

2.4 Môi trường dùng để ghi nhận kết quả 7

3 Kiến thức nền tảng **7**

3.1 Mô hình thực thi chương trình người dùng 7

3.2 Các system call liên quan 8

3.3 Các hàm thư viện người dùng 10

3.4 Process và giao tiếp qua pipe 11

3.5 Truy cập file system và duyệt thư mục 12

3.6 Xử lý dữ liệu và quản lý tài nguyên 13

4 Các chương trình đã thực hiện **14**

4.1 sleep 14

4.2 pingpong 15

4.3 primes 17

4.4 cp 18

4.5 tree 20

4.6 du 21

4.7 diff 23

5 Kết quả kiểm thử tổng hợp **25**

5.1 Môi trường và cách thực hiện 25

5.2 Kết quả theo từng chương trình 25
5.3 Nhật ký thực thi và phạm vi kết luận 26

6 Khó khăn gặp phải và giải pháp khắc phục 27

7 Kết luận và hướng phát triển 27

Tài liệu tham khảo 28

Tóm tắt báo cáo

Báo cáo này trình bày kết quả thực hiện, thiết kế kỹ thuật và quá trình kiểm thử toàn diện các chương trình tiện ích không gian người dùng (*User Programs*) trong khuôn khổ bài thực hành **Lab 01 (Hệ điều hành xv6 – Kiến trúc RISC-V)**. Mục tiêu trọng tâm của bài thực hành là tiếp cận kiến trúc phân tầng người dùng/kernel (*User/Kernel Space*), vận dụng các system call cơ bản, cơ chế quản lý vòng đời tiến trình, truyền thông liên tiến trình (IPC) và các thao tác truy xuất hệ thống tệp tin (*File System*).

Trong suốt quá trình thực hiện, nhóm đã phân tích yêu cầu, xây dựng mã giả và hiện thực thành công toàn bộ **07 chương trình tiện ích**:

- sleep: Tạm dừng tiến trình theo chu kỳ đồng hồ (*ticks*) của hệ thống.
- pingpong: Hiện thực cơ chế giao tiếp 2 chiều đồng bộ giữa tiến trình cha và con thông qua cặp đường ống (*pipes*).
- primes: Xây dựng thuật toán sàng số nguyên tố Eratosthenes đồng thời (*Concurrent Prime Sieve*) theo mô hình pipeline đa tiến trình kết nối liên tiếp từ 2 đến 280.
- cp: Sao chép dữ liệu tập tin an toàn qua bộ đệm (*buffer streaming*), xử lý kiểm tra trùng tệp nguồn/đích.
- tree: Duyệt đệ quy cây thư mục hiển thị phân cấp trực quan, hỗ trợ giới hạn độ sâu (-L) và lọc thư mục (-d).
- du: Ước tính dung lượng sử dụng đĩa (*Disk Usage*) theo byte, hỗ trợ tính tổng nhanh (-s) và liệt kê chi tiết tệp (-a).
- diff: So sánh nội dung hai tệp tin theo từng dòng, xử lý so khớp EOF và chế độ thông báo tóm tắt (-q).

Tóm tắt kết quả & Đánh giá chung:

- **Tỷ lệ hoàn thành:** 7/7 chương trình (100% khối lượng yêu cầu).
- **Độ tin cậy & Quản lý tài nguyên:** Tất cả các chương trình đều được kiểm thử nghiêm ngặt trên trình giả lập QEMU, kiểm soát chặt chẽ việc giải phóng bộ mô tả tập tin (không rò rỉ File Descriptor), đóng đúng các đầu pipe thừa để kích hoạt tín hiệu EOF chính xác, và đồng bộ hóa triệt để bằng wait() nhằm loại bỏ hoàn toàn nguy cơ sinh tiến trình ma (*zombie process*).
- **Kiểm thử tự động:** Đáp ứng đầy đủ các trường hợp kiểm thử chức

> năng và edge cases lỗi theo chuẩn kịch bản kiểm thử của môn học.

1. Thông tin thành viên và phân công công việc

STT	MSSV	Họ và tên	Nhiệm vụ phụ trách
1	24120438	Trần Nguyên Tân	sleep, pingpong, primes
2	24120023	Nguyễn Thành Bảo	cp, diff
3	24120040	Dương Hoài Đức	tree, du, tích hợp và kiểm thử chung

2. Thiết lập môi trường trên Windows và macOS

Do xv6 trong bài thực hành chạy trên kiến trúc RISC-V. Máy chủ đảm nhiệm việc biên dịch mã nguồn, còn QEMU mô phỏng phần cứng để khởi động xv6. Vì vậy, dù dùng Windows hay macOS, môi trường cần có trình biên dịch chéo RISC-V, bộ công cụ liên kết và QEMU; trình biên dịch C mặc định của máy không thay thế được bộ công cụ RISC-V.

2.1 Windows: cài đặt Ubuntu và các dependencies trên WSL 2

Trên Windows, quá trình biên dịch và kiểm thử được thực hiện trong Ubuntu trên WSL 2. Cách tổ chức này giữ được môi trường Linux theo yêu cầu bài thực hành mà vẫn cho phép soạn thảo tài liệu và quản lý mã nguồn từ Windows. Từ hướng dẫn của MIT, nhóm xác định sử dụng WSL 2 và QEMU từ phiên bản 7.2; Ubuntu phiên bản 24.04 để thuận tiện cài các gói phù hợp [6].

Cài đặt WSL. Mở PowerShell với quyền Administrator, cài Ubuntu rồi khởi động lại máy nếu được yêu cầu. Khi mở Ubuntu lần đầu, tạo tài khoản và mật khẩu Linux. Các lệnh dưới đây áp dụng cho Windows 10 phiên bản 2004 trở lên hoặc Windows 11 [9].

```
wsl --install -d Ubuntu-24.04
wsl --list --verbose
```

Cài công cụ phát triển. GCC và Binutils cho RISC-V dùng để tạo chương trình chạy trên xv6; build-essential cung cấp công cụ biên dịch phía máy chủ, QEMU chạy máy mô phỏng, còn Python dùng cho bộ kiểm thử của dự án. gdb-multiarch hỗ trợ gỡ lỗi khi cần.

```
sudo apt update
sudo apt install build-essential git perl python3 \
  gdb-multiarch qemu-system-misc \
  gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu

riscv64-linux-gnu-gcc --version
qemu-system-riscv64 --version
python3 --version
```

2.2 macOS: cài đặt công cụ RISC-V qua Homebrew

Trên macOS, có thể biên dịch trực tiếp trong Terminal và chạy xv6 bằng QEMU. Trước hết, cài Command Line Tools để có công cụ phát triển phía máy chủ, sau đó cài Homebrew theo hướng dẫn chính thức [10].

```
xcode-select --install
```

Sau khi cài Homebrew, thực hiện phần cấu hình brew shellenv được trình cài đặt hướng dẫn rồi mở lại Terminal. Đường dẫn này phụ thuộc vào cách cài và kiến trúc máy; không nên sao chép một đường dẫn cố định từ máy khác.

```
brew --version
brew install riscv64-elf-gcc qemu python

riscv64-elf-gcc --version
riscv64-elf-objdump --version
qemu-system-riscv64 --version
python3 --version
```

Gói riscv64-elf-gcc của Homebrew kéo theo Binutils tương ứng [11]. Đây là cách cài được dùng trong hướng dẫn này, thay cho cấu hình tap RISC-V trong tài liệu cũ. Khả năng cung cấp gói dựng sẵn phụ thuộc phiên bản macOS và kiến trúc máy, nên cần kiểm tra điều kiện hỗ trợ của Homebrew trước khi cài.

Bộ công cụ này có tiền tố riscv64-elf-, trong khi Makefile của mã nguồn hiện tại chưa tự nhận diện tiền tố đó. Vì vậy, khi biên dịch trên macOS phải truyền

rõ TOOLPREFIX, thay vì đổi tên các chương trình hoặc sửa cố định đường dẫn trong Makefile.

2.3 Chuẩn bị mã nguồn và khởi chạy xv6

Nhóm đã sử dụng mã nguồn trong folder xv6-labs-2024/ của dự án, source code của xv6 được clone từ git://g.csail.mit.edu/xv6-labs-2024

Từ thư mục gốc dự án, chạy các lệnh sau để biên dịch:

Ubuntu trên WSL 2

```
cd xv6-labs-2024
make
make qemu
```

macOS

```
cd xv6-labs-2024
make TOOLPREFIX=riscv64-elf-
make TOOLPREFIX=riscv64-elf- qemu
```

Môi trường sẵn sàng khi xv6 khởi động sẽ xuất hiện init: starting sh và dấu nhắc shell. Khi đó ta có thể chạy các lệnh sleep 10, pingpong hoặc primes để kiểm tra bước đầu. Để thoát QEMU, nhấn Ctrl+A, thả phím rồi nhấn X.

Các chương trình người dùng được khai báo trong UPROGS của Makefile. Ví dụ, tệp dựng user/_sleep được đưa vào fs.img với tên sleep; khi chạy trong shell xv6 không thêm dấu gạch dưới. Các tệp tạo trong xv6 thuộc ảnh đĩa này, không xuất hiện như tệp thông thường trong thư mục máy chủ. Lệnh make clean xóa sản phẩm biên dịch và ảnh đĩa, nên cần lưu dữ liệu thử nghiệm quan trọng trước khi dọn bản dựng.

Kiểm thử tự động. Từ thư mục gốc dự án, dùng python3 tests/lab01/run.py trên WSL. Trên macOS, thêm tùy chọn chọn bộ công cụ:

```
python3 tests/lab01/run.py --toolprefix=riscv64-elf-
```

2.4 Môi trường dùng để ghi nhận kết quả

Các thao tác kiểm thử được báo cáo ở mục 5 được thực hiện trong Ubuntu 24.04 qua WSL, với môi trường cài đặt GCC RISC-V 13.3.0 và QEMU 8.2.2. Đây là môi trường đã tạo ra kết quả 77/77 ca kiểm thử của báo cáo.

3. Kiến thức nền tảng

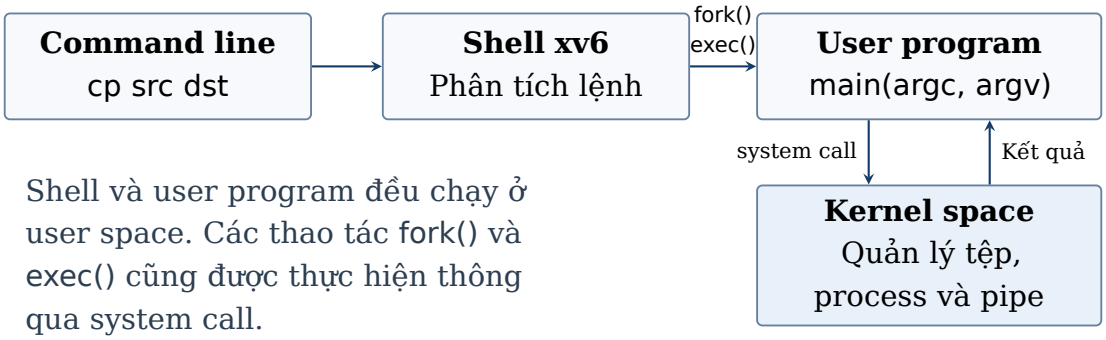
Để củng cố cơ sở lý thuyết cho các giải pháp trong bài thực hành, nhóm xin trình bày sơ lược mô hình thực thi user program, các system call và user library, cùng cơ chế quản lý process, pipe và file system của xv6. Phần trình bày tập trung vào những khái niệm và API được sử dụng trong bài, qua đó làm rõ cơ sở lựa chọn giải pháp và mối liên hệ giữa các cơ chế của xv6 với thiết kế từng lệnh ở mục 4.

3.1 Mô hình thực thi chương trình người dùng

Nắm được cách shell khởi chạy một user program giúp nhóm xác định phần việc cần xử lý trong chương trình và phần việc phải yêu cầu kernel thực hiện. Cách truyền command-line arguments cũng là cơ sở để thiết kế hàm main(), kiểm tra cú pháp và nhận đúng tham số cho các lệnh trong bài.

Khi người dùng nhập cp src dst, shell không tự thực hiện việc sao chép. Shell phân tích lệnh, tạo một child process bằng fork(), rồi gọi exec() trong process đó để nạp chương trình cp. Chương trình nhận các command-line arguments và thực hiện công việc được yêu cầu. Cách tổ chức này cho phép bổ sung một lệnh mới mà không phải đưa chức năng của lệnh vào shell hay kernel.

Vai trò của user space và kernel space. Shell và các chương trình trong bài đều chạy ở *user space*. Chúng xử lý tham số, điều khiển luồng thực thi và chuẩn bị dữ liệu, nhưng không trực tiếp truy cập các cấu trúc nội bộ của kernel. Những thao tác cần tài nguyên hệ thống, như mở tệp, đọc dữ liệu hoặc tạo process, được yêu cầu qua *system call*. Kernel xử lý yêu cầu trong *kernel space* rồi trả kết quả để chương trình tiếp tục.



Luồng khởi chạy một lệnh và tương tác giữa user program với kernel.

Phạm vi của user library. xv6 cung cấp một user library nhỏ, với các API được khai báo trong user/user.h. Những hàm như strcmp() hay atoi() hỗ trợ xử

lý dữ liệu ở user space; chúng không phải system call. Do thư viện này không có đầy đủ chức năng của thư viện C trên Linux hoặc macOS, chương trình phải sử dụng các API thực sự có trong xv6. Hai nhóm API được trình bày riêng trong các bảng ở phần tiếp theo.

3.2 Các system call liên quan

Để cài đặt các lệnh, nhóm cần chọn đúng system call cho từng thao tác và hiểu ý nghĩa của giá trị trả về. Kiến thức này được dùng khi yêu cầu tạm dừng trong sleep, tạo process và pipe trong pingpong, primes, cũng như mở, đọc và ghi tệp trong các lệnh còn lại. Các bảng dưới đây tóm tắt chức năng, tham số và kết quả cần kiểm tra của những system call liên quan.

Nhóm quản lý tiến trình và pipe.

System call	Chức năng và tham số	Kết quả và lưu ý
fork()	Tạo tiến trình con từ tiến trình hiện tại.	Trả 0 ở con, PID của con ở cha, -1 nếu thất bại. Cha và con cùng tiếp tục sau lời gọi.
wait(status)	Chờ và thu hồi một tiến trình con đã kết thúc; lưu mã thoát nếu truyền con trỏ status.	Trả PID của con được thu hồi, hoặc -1 nếu không có con để chờ hay tiến trình chờ bị hủy. Có thể dùng wait(0) khi không cần mã thoát.
exit(status)	Kết thúc tiến trình với mã trạng thái status.	Không quay lại nơi gọi. Kernel đóng các bộ mô tả tệp còn mở; cha nhận trạng thái qua wait().
getpid()	Lấy mã định danh của tiến trình đang chạy.	Trả PID; dùng để phân biệt cha và con trong kết quả pingpong.
sleep(ticks)	Tạm dừng tiến trình theo số tick được yêu cầu.	Trả 0 khi hoàn thành, -1 nếu bị hủy trong lúc chờ. Tick không phải đơn vị giây.
pipe(p)	Tạo kênh truyền một chiều; ghi hai descriptor vào mảng p.	Trả 0 khi thành công, -1 khi lỗi. p[0] là đầu đọc, p[1] là đầu ghi.

Nhóm truy cập tệp và nạp chương trình.

System call	Chức năng và tham số	Kết quả và lưu ý
open(path, mode)	Mở đường dẫn theo chế độ đọc, ghi hoặc tạo tệp.	Trả descriptor không âm, -1 nếu thất bại. Các cờ được định nghĩa trong kernel/fcntl.h.
close(fd)	Đóng descriptor không còn cần sử dụng.	Trả 0 khi thành công, -1 nếu descriptor không hợp lệ. Đóng một đầu pipe chỉ bỏ tham chiếu của tiến trình gọi.
read(fd, buf, n)	Đọc tối đa n byte từ tệp, thiết bị hoặc pipe vào vùng đệm.	Trả số byte đã đọc; 0 là hết dữ liệu (EOF), -1 là lỗi. Không tự thêm ký tự kết thúc chuỗi.
write(fd, buf, n)	Ghi n byte từ vùng đệm ra descriptor.	Trả số byte đã ghi hoặc -1 khi lỗi. Phải kiểm tra số byte thực tế, không mặc định luôn ghi đủ.
fstat(fd, st)	Lấy thông tin tệp đang mở vào struct stat.	Trả 0 khi thành công, -1 khi lỗi. Thông tin gồm loại, kích thước, thiết bị và số inode.
exec(path, argv)	Thay chương trình đang chạy bằng chương trình tại path, với danh sách tham số argv.	Giữ PID và các descriptor đang mở. Không quay lại khi thành công; trả -1 khi thất bại.

3.3 Các hàm thư viện người dùng

Các bước kiểm tra tham số, so sánh dữ liệu và cấp phát buffer được thực hiện bằng user library. Hiểu các hàm có sẵn và giới hạn của chúng giúp nhóm chọn cách xử lý phù hợp với xv6, từ chuyển đổi số trong sleep, nhận diện tùy chọn trong tree, du đến quản lý bộ nhớ cho dòng dài trong diff.

Không phải mọi hàm trong user/user.h đều là system call. Chẳng hạn, stat() là hàm thư viện thực hiện lần lượt open(), fstat() và close(). Phân biệt hai nhóm giúp xác định đúng vai trò của giao diện và tránh giả định rằng xv6 cung cấp đầy đủ thư viện C chuẩn.

Hàm thư viện	Công dụng	Điểm cần lưu ý
stat(path, st)	Lấy thông tin tệp theo đường dẫn.	Trả 0 khi thành công, -1 khi lỗi; descriptor mở bên trong được đóng trước khi trả về.
atoi(s)	Chuyển chuỗi chữ số thành số nguyên.	Không có cơ chế báo lỗi chuyển đổi riêng; chương trình phải kiểm tra định dạng đầu vào.
strcmp(a, b)	So sánh hai chuỗi kết thúc bằng ký tự NUL.	Trả 0 khi giống nhau; dùng để nhận diện tùy chọn và tên mục đặc biệt.
strlen(s)	Tính số ký tự trước NUL.	Không dùng để suy ra độ dài của dữ liệu nhị phân hoặc vùng đệm chưa kết thúc chuỗi.
memcmp(a, b, n)	So sánh đúng n byte của hai vùng nhớ.	Trả 0 khi bằng nhau; phù hợp để so sánh dòng theo độ dài, kể cả khi có byte NUL.
memmove(dst, src, n)	Sao chép n byte giữa hai vùng nhớ.	Cho phép hai vùng chồng lấn; cần bảo đảm vùng đích đủ lớn.
malloc(n), free(p)	Cấp phát và giải phóng bộ nhớ động.	Kiểm tra con trỏ trả về từ malloc(); giải phóng vùng nhớ sau khi dùng hoặc khi thay bộ đệm.
printf(fmt, ...), fprintf(fd, fmt, ...)	In thông tin theo định dạng.	printf() ghi ra descriptor 1; fprintf() ghi ra descriptor được chỉ định. Hai hàm trả về void trong xv6.

3.4 Process và giao tiếp qua pipe

Việc cài đặt pingpong và primes đòi hỏi xác định rõ process nào gửi, process nào nhận và khi nào mỗi bên được kết thúc. Hiểu cơ chế hoạt động của process và pipe là cơ sở để bố trí các kênh truyền, đóng đúng đầu pipe và chờ các process con, tránh tình trạng chương trình treo hoặc kết thúc khi công việc vẫn chưa hoàn tất.

Trong các lệnh pingpong và primes, công việc được chia cho nhiều process. Một process tạo process con bằng fork(); cả hai tiếp tục thực thi từ sau lời gọi này, với bộ nhớ riêng và các file descriptor được kế thừa. Cha hay con chạy trước phụ thuộc vào scheduler. Do đó, thứ tự xử lý phải được xác định bằng

việc trao đổi dữ liệu hoặc chờ, thay vì dựa vào thứ tự chạy tình cờ.

Pipe là kênh truyền dữ liệu một chiều giữa các process, gồm một đầu đọc và một đầu ghi. Dữ liệu bên gửi ghi vào được lưu trong buffer của kernel để bên nhận đọc ra. Nếu chưa có dữ liệu nhưng đầu ghi vẫn mở, bên nhận sẽ chờ; nếu buffer đầy, bên gửi có thể phải chờ bên nhận đọc bớt. Với trao đổi hai chiều như pingpong, mỗi chiều dùng một pipe riêng.

Việc đóng đầu pipe quyết định khi nào quá trình nhận dữ liệu kết thúc. Sau fork(), cha và con đều giữ các đầu pipe đã mở, nên mỗi bên phải đóng những đầu không dùng. read() chỉ trả về EOF khi buffer đã được đọc hết và không còn process nào giữ đầu ghi. Một đầu ghi bị bỏ quên có thể khiến bên nhận tiếp tục chờ dù bên gửi đã hoàn thành. Đây cũng là điều kiện kết thúc mỗi bước sàng trong primes.

Sau khi gửi hết dữ liệu, process cha vẫn cần gọi wait() để chờ con xử lý xong và thu hồi thông tin của con khi kết thúc. Còn sleep() tạm dừng process theo số tick được yêu cầu, nhường CPU cho các công việc khác. Hết khoảng chờ, process được đưa về trạng thái sẵn sàng; scheduler quyết định thời điểm nó tiếp tục chạy.

3.5 Truy cập file system và duyệt thư mục

Các lệnh cp, tree, du và diff cần truy cập đúng tệp và xác định được loại dữ liệu đang xử lý. Kiến thức về file descriptor, inode và cấu trúc thư mục giúp nhóm chọn chế độ mở tệp, kiểm tra nguồn và đích của cp, đồng thời xây dựng cách duyệt đệ quy và cộng dung lượng cho tree, du.

Chương trình truy cập một tệp bằng cách mở đường dẫn rồi dùng file descriptor do kernel trả về cho các lần đọc, ghi tiếp theo. File descriptor là một số nguyên có ý nghĩa trong process đang sử dụng nó. Giao diện này cũng áp dụng cho pipe và thiết bị, nên chương trình có thể dùng cùng read() và write() với những nguồn dữ liệu khác nhau. Theo quy ước của shell, descriptor 0, 1 và 2 tương ứng với standard input, standard output và standard error.

Để biết một đường dẫn là tệp thường hay thư mục, chương trình lấy metadata bằng stat() hoặc fstat(). Kết quả được lưu trong struct stat, trong đó type cho biết loại tệp và size cho biết độ dài nội dung theo byte. Tên tệp và bản thân tệp là hai khái niệm khác nhau: một tên trong thư mục trỏ đến inode lưu thông tin về tệp, và nhiều tên có thể cùng trỏ đến một inode. Vì vậy, cp kiểm tra cặp dev, ino của nguồn và đích trước khi ghi đè, thay vì chỉ so sánh hai đường dẫn.

Chế độ đọc, ghi và cách xử lý tệp đã tồn tại được chọn qua các cờ của open(). Những cờ dùng trong bài có ý nghĩa như sau:

Cờ	Ý nghĩa
O_RDONLY	Mở chỉ đọc.
O_WRONLY	Mở chỉ ghi.
O_RDWR	Mở cho cả đọc và ghi.
O_CREATE	Tạo tệp nếu chưa tồn tại; không tự xóa nội dung của tệp đã có.
O_TRUNC	Đưa độ dài tệp thường về 0 khi mở; cần kiểm tra nguồn và đích trước khi dùng.

Khi duyệt thư mục, chương trình đọc các directory entry chứa tên và số inode của từng mục con, bỏ qua entry chưa sử dụng rồi ghép đường dẫn để tiếp tục truy cập. Hai mục . và .. phải được loại khỏi quá trình duyệt xuống vì chúng trỏ đến thư mục hiện tại và thư mục cha. Tên trong entry có độ dài cố định và có thể thiếu ký tự kết thúc NUL, nên cần chuyển sang chuỗi C hợp lệ trước khi xử lý.

Cấu trúc thư mục phù hợp với cách duyệt đệ quy: gặp tệp thường thì xử lý trực tiếp, gặp thư mục thì áp dụng lại cách duyệt cho các mục bên trong. tree theo dõi độ sâu để hiển thị phân cấp và giới hạn việc đi xuống. du cộng kết quả từ các thư mục con để tính tổng. Tổng được dùng trong bài là độ dài nội dung các tệp theo byte, không tính dung lượng của bản thân thư mục hay số khối đĩa thực tế đã cấp phát.

3.6 Xử lý dữ liệu và quản lý tài nguyên

Đọc đúng dữ liệu và thu dọn tài nguyên là yêu cầu chung khi cài đặt các chương trình trong bài. Nắm được cách dùng buffer, nhận biết EOF và kiểm tra lỗi giúp nhóm xử lý tệp lớn trong cp, dòng dài trong diff, cũng như xác định những tệp, vùng nhớ và process cần được thu hồi khi hoàn thành hoặc gặp lỗi.

Các tệp có thể lớn hơn nhiều so với vùng nhớ dành cho việc xử lý. Vì vậy, cp đọc và ghi từng phần qua một buffer, không nạp toàn bộ tệp vào bộ nhớ. Mỗi lượt đọc trả về số byte thực tế nhận được; lượng dữ liệu đem ghi phải dựa trên con số này, nhất là ở cuối tệp khi buffer thường không còn đầy. Giá trị 0 cho biết đã hết dữ liệu, còn -1 báo lỗi đọc. Khi ghi, chương trình cũng cần kiểm tra số byte đã ghi thay vì mặc định thao tác luôn hoàn tất.

Với diff, dữ liệu được tổ chức thành dòng để so sánh. Một dòng có thể dài hơn buffer ban đầu, và dòng cuối vẫn hợp lệ dù không có ký tự xuống dòng. Dòng rỗng cũng phải được phân biệt với EOF: dòng rỗng có ký tự xuống dòng, trong khi EOF cho biết không còn dữ liệu để đọc. Những trường hợp này cần được xử lý ngay trong bước đọc dòng để kết quả so sánh không bị sai lệch.

Kích thước buffer phải đủ cho cả dữ liệu và phần kết thúc chuỗi nếu dùng chuỗi C. Với đường dẫn, độ dài cần tính cả thư mục hiện tại, dấu phân cách, tên mục con và NUL. Vùng nhớ động phải được kiểm tra sau khi cấp phát và giải phóng khi không còn dùng. Tương tự, tệp và đầu pipe được đóng sau khi xử lý; process cha chờ con nếu cần bảo đảm công việc của con đã hoàn tất.

Khi gặp lỗi, chương trình ghi thông báo vào standard error, kèm thao tác hoặc đường dẫn liên quan. Exit status được dùng để trả kết quả cho process cha. Riêng diff, mã 0 biểu thị hai tệp giống nhau, mã 1 biểu thị có khác biệt và mã 2 biểu thị lỗi thực thi. Việc hai tệp khác nhau vì thế không được xem là lỗi của chương trình.

Trong bản hiện thực này, các vòng đọc của primes, tree và du chưa phân biệt riêng lỗi đọc với trường hợp không đọc đủ một phần tử. du cũng trả 0 khi không truy cập được một nhánh, nên tổng có thể thiếu dữ liệu nếu xảy ra lỗi. Các kết quả kiểm thử ở mục 5 cần được hiểu trong phạm vi đó.

4. Các chương trình đã thực hiện

4.1 sleep

Mục tiêu.

- **Chức năng:** Tạm dừng tiến trình hiện tại trong số chu kỳ đồng hồ do ticks chỉ định.
- **Tham số:** Chấp nhận đúng một tham số bắt buộc theo cú pháp sleep ticks; ticks phải là một giá trị số hợp lệ.
- **Xử lý lỗi:** Phát hiện trường hợp thiếu, thừa hoặc sai định dạng tham số và thông báo lỗi phù hợp.

Kiến thức sử dụng.

- **Tick và thời gian hệ thống:** Tick là đơn vị đếm theo nhịp ngắt của bộ định thời trong xv6. Số tick cho biết khoảng cần chờ, nhưng không thể hiểu trực tiếp là số giây.
- **Trạng thái chờ của tiến trình:** Một tiến trình có thể tạm ngừng thực thi cho đến khi điều kiện chờ được đáp ứng. System call sleep() sử dụng cơ chế này, để tiến trình nhường CPU thay vì liên tục kiểm tra đồng hồ.
- **Chuyển đổi tham số số:** Tham số dòng lệnh ban đầu là chuỗi ký tự, kể cả khi người dùng nhập một số. Kiểm tra chuỗi trước khi chuyển đổi giúp phân

biệt giá trị số với dữ liệu sai định dạng.

Ý tưởng triển khai. Lệnh sleep sử dụng cơ chế chờ của hệ điều hành thay vì tự đếm thời gian bằng một vòng lặp. Sau khi kiểm tra tham số không rỗng và chỉ chứa chữ số, chương trình chuyển giá trị nhập thành số tick rồi yêu cầu tạm dừng tiến trình. Trong thời gian đó, tiến trình không chiếm CPU để chờ, nên hệ thống có thể chạy các công việc khác. Hết khoảng chờ, tiến trình được phép chạy tiếp và kết thúc. Thời lượng ở đây được tính theo tick của xv6, không phải theo giây.

Mã giả dưới đây mô tả các bước kiểm tra tham số và tạm dừng tiến trình trong hàm main().

sleep

```
1  PROCEDURE Main(arguments)
2      IF arguments do not contain exactly one value THEN
3          REPORT usage error; EXIT 1
4      END IF
5      value ← the supplied argument
6      IF value is empty or contains a non-digit character THEN
7          REPORT invalid duration; EXIT 1
8      END IF
9      ticks ← ConvertToInteger(value)
10     Suspend the current process for ticks clock ticks
11     EXIT 0
12 END PROCEDURE
```

4.2 pingpong

Mục tiêu.

- **Chức năng:** Minh họa giao tiếp hai chiều giữa tiến trình cha và tiến trình con bằng cách gửi “ping” và nhận phản hồi “pong”.
- **Tham số:** Không nhận tham số dòng lệnh; lệnh được thực thi trực tiếp bằng pingpong.
- **Kết quả:** In thông báo giao tiếp kèm PID của các tiến trình và kết thúc bình thường sau khi trao đổi hoàn tất.

Kiến thức sử dụng.

- **Tiến trình cha và con:** Sau fork(), cha và con tiếp tục chạy độc lập từ cùng vị trí nhưng nhận giá trị trả về khác nhau. Nhờ đó, chương trình phân chia được vai trò gửi và phản hồi cho hai tiến trình.

- **Pipe một chiều:** Một pipe chỉ truyền dữ liệu theo một hướng, từ đầu ghi đến đầu đọc. Lượt trao đổi cần cả gửi lẫn phản hồi nên sử dụng hai pipe, mỗi pipe đảm nhiệm một chiều.
- **Đồng bộ tiến trình:** Thao tác đọc pipe có thể chờ khi chưa có dữ liệu, còn wait() cho phép cha chờ con kết thúc. Hai cơ chế này xác lập quan hệ trước–sau giữa các công việc mà không phụ thuộc tiến trình nào được cấp CPU trước.
- **Các đầu pipe được kế thừa:** Tiến trình con thừa hưởng các đầu pipe đã mở trước khi được tạo. Mỗi bên cần đóng những đầu không dùng và chỉ giữ đầu đọc, đầu ghi phù hợp với vai trò của mình.

Ý tưởng triển khai. Hai tiến trình trao đổi qua hai pipe, một pipe cho chiều cha gửi sang con và một pipe cho chiều phản hồi. Tiến trình cha gửi một byte để bắt đầu lượt trao đổi. Khi nhận được byte này, tiến trình con in thông báo nhận ping, gửi lại byte đó rồi kết thúc. Cha chờ con hoàn tất, đọc phản hồi và in thông báo nhận pong. Nhờ trình tự gửi–nhận này, thông báo ping luôn xuất hiện trước pong mà không cần quy định tiến trình nào phải được chạy trước. Các đầu pipe không sử dụng được đóng ở mỗi bên; những đầu còn lại cũng được đóng sau khi trao đổi xong.

Trình tự trao đổi giữa tiến trình cha và con được trình bày trong mã giả dưới đây, tương ứng với hàm main().

pingpong

```
1  PROCEDURE Main()
2      Create channels p (parent to child) and q (child to parent)
3      Create a child process
4      IF channel creation or process creation fails THEN
5          REPORT error; EXIT 1
6      END IF
7      IF executing in the parent THEN
8          Close unused channel ends
9          Send one byte through p; close the sending end
10         Wait for the child to finish
11         Receive one byte through q
12         OUTPUT parent identifier and "received pong"
13         Close the receiving end
14     ELSE
15         Close unused channel ends
16         Receive one byte through p
17         OUTPUT child identifier and "received ping"
18         Send the same byte through q
19         Close the remaining channel ends
20     END IF
```

```
    On any send or receive failure, report error and exit 1
    EXIT 0
END PROCEDURE
```

4.3 primes

Mục tiêu.

- **Chức năng:** Sinh và in các số nguyên tố trong phạm vi từ 2 đến 280.
- **Tham số:** Không nhận tham số dòng lệnh; phạm vi xử lý được cố định theo yêu cầu của bài thực hành.
- **Mục tiêu xử lý:** Thể hiện mô hình sàng Eratosthenes đồng thời và bảo đảm mỗi số nguyên tố được xuất đúng một lần.

Kiến thức sử dụng.

- **Sàng số nguyên tố:** Mọi hợp số đều có ít nhất một ước nguyên tố nhỏ hơn nó. Loại dần các bội của những số nguyên tố đã tìm được sẽ giữ lại các ứng viên cho bước sàng tiếp theo.
- **Pipeline tiến trình:** Pipeline chia công việc thành các bước nối tiếp, trong đó đầu ra của bước trước là đầu vào của bước sau. Với bài này, mỗi tiến trình đảm nhiệm một bộ lọc và truyền số qua pipe, không cần dùng chung một mảng dữ liệu.
- **EOF và kết thúc chuỗi xử lý:** Bên đọc nhận EOF khi dữ liệu trong pipe đã hết và mọi đầu ghi đã đóng; vì vậy, mỗi bước chỉ giữ những đầu pipe cần dùng. Việc chờ tiến trình con hoàn tất giúp thu hồi thông tin tiến trình đã kết thúc và bảo đảm phần việc phía sau hoàn thành trước khi bước hiện tại thoát.

Ý tưởng triển khai. Các số từ 2 đến 280 được đưa lần lượt qua một chuỗi tiến trình, mỗi tiến trình thực hiện một bước sàng. Bước đầu lấy 2 làm số nguyên tố và loại các số chia hết cho 2. Những số còn lại được chuyển sang bước sau, nơi 3 được chọn và các bội của 3 tiếp tục bị loại. Cách xử lý này lặp lại cho đến khi không còn số cần xét. Số đầu tiên ở mỗi bước là số nguyên tố: nếu nó chia hết cho một số nguyên tố nhỏ hơn, nó đã bị loại ở bước sàng trước đó.

Các bước sàng nối với nhau bằng pipe và xử lý từng số khi nhận được, không cần gom toàn bộ dãy còn lại vào bộ nhớ. Tiến trình kế tiếp chỉ được tạo khi có số vượt qua bước lọc hiện tại. Sau khi gửi hết dữ liệu, tiến trình đóng đầu ghi để báo hết dữ liệu cho bước sau, rồi chờ bước đó hoàn tất trước khi thoát.

Mã giả dưới đây mô tả một bước sàng trong hàm primes(left). Các bước tiếp theo áp dụng cùng cách xử lý cho những số còn lại.

primes

```
PROCEDURE Primes(left)
    prime ← ReceiveInteger(left)
    IF a complete integer was not received THEN
        Close left; EXIT 0
    END IF
    OUTPUT "prime" followed by prime
    right ← no outgoing channel
    WHILE a complete integer n can be received from left DO
        IF n is divisible by prime THEN
            CONTINUE
        END IF
        IF right has not been created THEN
            Create a channel and a child process
            IF creation fails THEN
                REPORT error; EXIT 1
            END IF
            Child closes unused ends and runs Primes(new channel)
            Parent keeps the sending end as right
        END IF
        Send n through right
        IF sending fails THEN
            REPORT error; EXIT 1
        END IF
    END WHILE
    Close left
    IF right exists THEN
        Close right; wait for the child to finish
    END IF
    EXIT 0
END PROCEDURE
```

4.4 cp

Mục tiêu.

- **Chức năng:** Sao chép toàn bộ nội dung từ tệp nguồn src sang tệp đích dst.
- **Tham số:** Nhận đúng hai tham số đường dẫn theo cú pháp cp src dst.
- **Xử lý lỗi:** Kiểm tra nguồn không hợp lệ, đích không thể tạo hoặc mở, trường hợp nguồn và đích cùng trỏ tới một tệp, đồng thời báo lỗi khi cú pháp không đúng.

Kiến thức sử dụng.

- **Bộ mô tả tệp:** Khi mở một tệp, tiến trình nhận một số nguyên dùng để truy cập tệp đó trong các thao tác đọc, ghi và đóng. Hai tệp nguồn và đích được mở với chế độ khác nhau, phù hợp với chiều sao chép.
- **Đọc–ghi theo khối:** Bộ đệm giữ một phần dữ liệu trong lúc chuyển từ nguồn sang đích, nên không cần nạp toàn bộ tệp vào bộ nhớ. Số byte thực sự đọc được quyết định lượng cần ghi, đặc biệt ở phần cuối tệp.
- **Nhận diện tệp và liên kết cứng:** Hai đường dẫn khác nhau có thể trỏ đến cùng một tệp thông qua liên kết cứng. Cặp thông tin thiết bị và số inode, tức mã nhận diện tệp trên thiết bị, cho phép phát hiện trường hợp này trước khi ghi đè nội dung.

Ý tưởng triển khai. Trước khi sao chép, cần xác định nguồn và đích có phải là hai tệp khác nhau hay không. Chỉ so sánh đường dẫn là chưa đủ, bởi hai tên có thể cùng trỏ đến một tệp. Chương trình vì vậy kiểm tra thông tin nhận diện của tệp và loại tệp trước khi xóa nội dung đích. Nguồn phải là tệp thông thường; nếu đích đã tồn tại thì đích cũng phải đáp ứng điều kiện này.

Dữ liệu sau đó được đọc từ nguồn và ghi sang đích theo từng khối tối đa 512 byte. Mỗi lượt ghi đúng lượng dữ liệu vừa đọc được, nên phần cuối của tệp vẫn được sao chép đầy đủ dù ngắn hơn một khối. Việc sao chép tuần tự giữ nguyên thứ tự các byte và không đòi hỏi bộ nhớ tăng theo kích thước tệp. Khi gặp lỗi đọc hoặc ghi, chương trình dừng và báo lỗi; dữ liệu đã ghi vào đích trước đó không được khôi phục.

Quy trình kiểm tra tệp và sao chép dữ liệu trong hàm main() được tóm tắt bằng mã giả dưới đây.

cp

```
1  PROCEDURE Main(arguments)
2      Require exactly two arguments: source and destination
3      Open source and verify that it is a regular file
4      IF destination already exists THEN
5          Verify that it is a different regular file
6      END IF
7      IF an argument or file check fails THEN
8          Close opened files; report error; EXIT 1
9      END IF
10     Create or open destination, replacing its old contents
11     IF destination cannot be opened THEN
12         Close source; report error; EXIT 1
13     END IF
14     WHILE reading source yields a nonempty portion DO
```

```
        Write exactly that portion to destination
        IF the full portion was not written THEN
            Close both files; report error; EXIT 1
        END IF
    END WHILE
    IF the last read failed THEN
        Close both files; report error; EXIT 1
    END IF
    Close both files
    EXIT 0
END PROCEDURE
```

4.5 tree

Mục tiêu.

- **Chức năng:** Hiển thị thư mục và tệp tin theo dạng cây phân cấp.
- **Tham số:** Nhận đường dẫn tùy chọn path, mặc định là thư mục hiện hành .; hỗ trợ -L depth để giới hạn độ sâu và -d để chỉ liệt kê thư mục.
- **Xử lý lỗi:** Phát hiện và thông báo rõ ràng các đường dẫn, tham số hoặc tùy chọn không hợp lệ.

Kiến thức sử dụng.

- **Cây thư mục:** Một thư mục chứa các mục trỏ đến tệp hoặc thư mục con, tạo nên cấu trúc phân cấp. Biết loại của từng mục giúp xác định nhánh nào có thể tiếp tục duyệt; trong xv6, thông tin theo đường dẫn được lấy qua hàm thư viện stat().
- **Đệ quy và duyệt chiều sâu:** Mỗi thư mục con có thể được xử lý theo cùng cách với thư mục chứa nó. Đệ quy cho phép đi sâu vào một nhánh rồi quay lại các mục còn lại, đồng thời dùng độ sâu hiện tại để xác định mức thụt lề.
- **Điều kiện dừng khi duyệt:** Hai mục . và .. trỏ đến thư mục hiện tại và thư mục cha, nên không được xem là nhánh mới. Giới hạn độ sâu là một điều kiện dừng khác, cho phép chỉ duyệt đến số tầng người dùng yêu cầu.

Ý tưởng triển khai. Từ thư mục được chọn, chương trình duyệt theo chiều sâu: in tên một mục, duyệt hết nội dung bên trong nếu đó là thư mục, rồi mới chuyển sang mục kế tiếp. Mức thụt lề tăng theo độ sâu, thể hiện vị trí của từng mục trong cây. Hai mục . và .. được bỏ qua để không quay lại thư mục hiện tại hoặc thư mục cha; các mục chưa sử dụng cũng không được đưa vào kết quả.

Tùy chọn -L giới hạn số tầng được hiển thị, còn -d chỉ giữ lại các thư mục. Khi

đạt độ sâu cho phép, chương trình không duyệt sâu hơn nhánh đó. Nếu đường dẫn đầu vào là một tệp thay vì thư mục, kết quả chỉ gồm tên tệp, trừ trường hợp đang bật -d.

Mã giả dưới đây thể hiện cách duyệt và hiển thị một nhánh thư mục, dựa trên hàm tree_traverse(path, level).

tree

```
PROCEDURE TreeTraverse(path, level)
    IF a depth limit is set and level reaches it THEN
        RETURN
    END IF
    Open directory path
    IF opening fails THEN
        REPORT error; RETURN
    END IF
    IF child paths would exceed the allowed length THEN
        Close directory; report error; RETURN
    END IF
    FOR EACH entry read from directory DO
        IF entry is unused, "." or ".." THEN
            CONTINUE
        END IF
        childPath ← Join(path, entry.name)
        IF information for childPath cannot be obtained THEN
            REPORT error; CONTINUE
        END IF
        IF directories-only mode excludes entry THEN
            CONTINUE
        END IF
        Display entry.name with indentation for level + 1
        IF entry is a directory THEN
            TreeTraverse(childPath, level + 1)
        END IF
    END FOR
    Close directory
END PROCEDURE
```

4.6 du

Mục tiêu.

- **Chức năng:** Tính và hiển thị dung lượng sử dụng của tệp hoặc thư mục theo đơn vị byte.
- **Tham số:** Nhận đường dẫn tùy chọn path, mặc định là .; hỗ trợ -s để chỉ in

tổng dung lượng và -a để hiển thị thêm dung lượng của từng tệp.

- **Đầu ra:** Hiển thị dung lượng theo byte cùng đường dẫn tương ứng và tuân theo chế độ hiển thị đã chọn.

Kiến thức sử dụng.

- **Thông tin kích thước tệp:** Hệ thống tệp lưu kích thước của từng tệp trong thông tin quản lý của nó. Lấy giá trị này đủ để thống kê số byte, không cần đọc lại toàn bộ nội dung tệp.
- **Duyệt hậu thứ tự:** Khi kết quả của thư mục cha phụ thuộc các mục con, các mục con phải được tính trước. Cách duyệt này cho phép mỗi nhánh trả về một tổng để cộng vào thư mục chứa nó.
- **Kích thước dữ liệu và dung lượng đĩa:** Số byte của tệp không đồng nhất với số khối đĩa được cấp phát để lưu tệp. Phép tính trong bài cộng kích thước các tệp theo byte, không cộng kích thước bản thân thư mục hay quy đổi sang dung lượng khối đĩa.

Ý tưởng triển khai. Dung lượng của một thư mục được tính từ các tệp và thư mục con bên trong. Với tệp thông thường hoặc tệp thiết bị, chương trình lấy kích thước do hệ thống ghi nhận. Với thư mục con, chương trình tính xong tổng của nhánh đó rồi cộng vào thư mục cha. Kết quả vì vậy được tổng hợp từ dưới lên: các mục con hoàn tất trước, thư mục chứa chúng hoàn tất sau.

Các tùy chọn chỉ thay đổi phần kết quả được in, không thay đổi cách cộng dung lượng. Chế độ mặc định in tổng của từng thư mục; -a in thêm từng tệp, còn -s chỉ in tổng của đường dẫn được chọn. Trong bài này, dung lượng là tổng kích thước theo byte, không tính kích thước bản thân thư mục hay số khối đĩa thực tế sử dụng. Nhánh không truy cập được sẽ được báo lỗi và bỏ qua khi tính tổng.

Cách tính dung lượng và cộng dồn kết quả từ các mục con được trình bày trong mã giả của hàm du(path) dưới đây.

du

```
1  PROCEDURE Du(path)
2      Obtain information about path
3      IF information cannot be obtained THEN
4          REPORT error; RETURN 0
5      END IF
6      IF path is a regular file or a device THEN
7          IF all-files mode is enabled and summary mode is off THEN
8              OUTPUT size in bytes and path
9          END IF
10         RETURN size in bytes
11     END IF
```

```
    IF path is not a directory THEN
        RETURN 0
    END IF
    Open directory and check the allowed child-path length
    IF opening or the length check fails THEN
        Close directory if opened; report error; RETURN 0
    END IF
    total ← 0
    FOR EACH entry read from directory DO
        IF entry is unused, "." or ".." THEN
            CONTINUE
        END IF
        total ← total + Du(Join(path, entry.name))
    END FOR
    Close directory
    IF summary mode is off THEN
        OUTPUT total and path
    END IF
    RETURN total
END PROCEDURE
```

4.7 diff

Mục tiêu.

- **Chức năng:** So sánh nội dung của hai tệp theo từng dòng và xác định các vị trí khác biệt.
- **Tham số:** Nhận hai tệp bắt buộc file1, file2 và tùy chọn -q theo cú pháp diff file1 file2 [-q].
- **Đầu ra:** Ở chế độ thông thường, chỉ ra các dòng khác biệt của từng tệp; với -q, chỉ thông báo hai tệp có khác nhau hay không.

Kiến thức sử dụng.

- **So sánh theo vị trí:** Hai dòng được đối chiếu khi có cùng số thứ tự trong hai tệp. Đây là phép so sánh từng cặp, không tìm cách ghép lại các dòng tương ứng khi có chèn hoặc xóa nội dung.
- **Dòng trống, EOF và dấu xuống dòng:** Dòng trống vẫn là một dòng, còn EOF cho biết không còn dữ liệu để đọc. Dấu xuống dòng cũng là một phần cần kiểm tra, vì dòng cuối có thể kết thúc ngay tại EOF mà không có dấu này.
- **Bộ đệm động:** Độ dài dòng không biết trước, nên vùng nhớ chứa dòng cần

được mở rộng khi chưa đủ chỗ. Theo dõi số byte thực tế giúp giữ đủ nội dung, kể cả khi có byte 0 nằm giữa dòng; vùng nhớ được giải phóng sau khi sử dụng.

Ý tưởng triển khai. Hai tệp được đọc song song và so sánh các dòng có cùng số thứ tự. Ngoài nội dung, chương trình kiểm tra độ dài và dấu xuống dòng, đồng thời phân biệt dòng trống với trường hợp tệp đã hết dữ liệu. Do đó, hai dòng có cùng phần chữ nhưng khác dấu xuống dòng cuối tệp vẫn được xem là khác nhau. Bộ đệm được mở rộng theo độ dài dòng để không cắt mất phần nội dung cần so sánh.

Với mỗi khác biệt, kết quả ghi tên tệp, số dòng và nội dung của hai phía. Nếu một tệp kết thúc trước, các dòng còn lại ở tệp kia được đối chiếu với trạng thái hết tệp. Khi có -q, chương trình dừng ở khác biệt đầu tiên và chỉ in thông báo. Phép so sánh trong bài không căn chỉnh lại dòng khi có chèn hoặc xóa: một dòng thêm vào giữa tệp có thể khiến nhiều cặp dòng phía sau được báo khác nhau.

Mã giả dưới đây mô tả quy trình so sánh trong hàm main(), từ việc đọc từng cặp dòng đến ghi nhận và hiển thị khác biệt.

diff

```
1  PROCEDURE Main(arguments)
2      Parse two paths and optional quiet flag at either end
3      Reject invalid syntax or paths beginning with a hyphen
4      Open both files and verify that they are regular files
5      On a setup failure, close opened files and exit 2
6      status ← 0; number ← 1
7      LOOP
8          first ← ReadLine(firstFile)
9          second ← ReadLine(secondFile)
10         IF either line could not be read THEN
11             REPORT error; status ← 2; BREAK
12         END IF
13         IF neither file has another line THEN
14             BREAK
15         END IF
16         IF presence, length, newline or contents differ THEN
17             status ← 1
18             IF quiet mode is enabled THEN
19                 OUTPUT "diff: files differ"; BREAK
20             END IF
21             PrintLine(firstFile, number, first)
22             PrintLine(secondFile, number, second)
23             IF either output operation fails THEN
24                 REPORT error; status ← 2; BREAK
```

```
25            END IF
26          END IF
27          number ← number + 1
28        END LOOP
29        Release stored lines and close both files
30        EXIT status
31    END PROCEDURE
```

5. Kết quả kiểm thử tổng hợp

5.1 Môi trường và cách thực hiện

Kiểm thử được thực hiện tự động trên xv6 chạy bằng QEMU 8.2.2, với 128 MB bộ nhớ và 3 CPU mô phỏng. Mã nguồn được biên dịch trên Ubuntu 24.04 trong WSL bằng GCC 13.3.0 cho RISC-V. Bộ chạy kiểm thử tạo một bản sao tạm của xv6, biên dịch lại chương trình và tạo ảnh hệ thống tệp riêng; vì vậy, các tệp thử nghiệm không làm thay đổi fs.img trong thư mục làm việc.

Bộ kiểm thử gồm cptest, difftest, linetest và coretest, đặt tại tests/lab01/. Các bài kiểm tra đối chiếu kết quả với giá trị mong đợi, bao gồm nội dung tệp, thông báo xuất ra và mã thoát. Riêng sleep, thời gian chờ được đo bằng bộ đếm tick của xv6. Lệnh chạy từ thư mục gốc của dự án, trong Ubuntu WSL, là:

```
python3 tests/lab01/run.py \
  --log docs/test-results/lab01-qemu.log
```

5.2 Kết quả theo từng chương trình

Lần chạy ghi nhận **77/77 phép kiểm tra đạt**, không xuất hiện thông báo FAIL và bộ chạy kiểm thử kết thúc với mã 0. Con số này bao gồm các lần chạy lặp lại và một kiểm tra tạo liên kết cứng phục vụ thử nghiệm cp; không phải số lượng trường hợp độc lập hay tỷ lệ bao phủ mã nguồn.

Đối tượng	Phạm vi kiểm tra	Kết quả ghi nhận	Đạt/tổng
sleep	Chờ 5 tick, 0 tick; thiếu, thừa, sai định dạng và số âm	Lượt sleep 5 đo được 5 tick; thông báo và mã thoát đúng kỳ vọng	6/6
pingpong	Chạy 3 lần; kiểm tra thứ tự thông báo và PID	Mỗi lần in ping trước pong; hai PID khác nhau; mã thoát 0	3/3
primes	Chạy 2 lần; đối chiếu dãy bằng phép thử chia độc lập	Đủ 59 số nguyên tố từ 2 đến 280, đúng thứ tự, không thừa hoặc trùng	2/2
cp	Tệp 0–4097 byte; ghi đè; cùng tệp, liên kết cứng; đối số và đường dẫn lỗi	Đúng từng byte và kích thước; giữ nguyên nguồn trong các ca từ chối sao chép	18/18
tree	Cây nhiều tầng; -d, -L 1, -L 2; tệp, thư mục rỗng và đầu vào lỗi	Cấu trúc, thụt lề, giới hạn độ sâu, thông báo và mã thoát đúng kỳ vọng	11/11
du	Tổng thư mục; -a, -s, kết hợp hai cờ; tệp đơn và đầu vào lỗi	Tệp mẫu 3, 5, 2 byte cho tổng 10 byte; kết quả chi tiết và tóm tắt đúng	8/8
diff	Tệp rỗng; dòng khác nhau; -q; hết tệp, dấu xuống dòng; dòng dài và byte 0	Nội dung xuất và mã thoát 0/1/2 đúng kỳ vọng, kể cả dòng 1200 byte	20/20
Đọc dòng	Dòng trống, dòng cuối, hết tệp, mở rộng bộ đệm, byte 0 và lỗi đọc	Phân biệt dòng trống với hết tệp; giữ đủ nội dung và nhận biết lỗi đọc	9/9
Tổng		**Tất cả phép kiểm tra đều đạt**	**77/77**

5.3 Nhật ký thực thi và phạm vi kết luận

Nhật ký QEMU đầy đủ được lưu tại docs/test-results/lab01-qemu.log. Trích các dòng xác nhận hoàn tất từ lần chạy thực tế:

```
ALL CP TESTS PASSED
ALL DIFF TESTS PASSED
ALL LINE TESTS PASSED
MEASURE: sleep 5 elapsed=5 ticks
ALL SLEEP TESTS PASSED
```

```
ALL PINGPONG TESTS PASSED
ALL PRIMES TESTS PASSED
ALL TREE TESTS PASSED
ALL DU TESTS PASSED
ALL CORE TESTS PASSED
```

Các kết quả trên xác nhận hành vi của chương trình trong những trường hợp đã kiểm tra, không chứng minh chương trình không còn lỗi. Bộ test chưa mô phỏng đầy đĩa, thiếu bộ nhớ, cạn tài nguyên tiến trình hoặc mọi giới hạn đường dẫn; mẫu sao chép lớn nhất là 4097 byte và mẫu dòng dài là 1200 byte. Thời gian 5 tick là số đo của lượt chạy này, không phải cam kết tiến trình luôn được lập lịch lại đúng thời điểm trên mọi lần chạy.

Đây là **bộ kiểm thử riêng cho phạm vi Lab 01 của báo cáo**, không phải kết quả make grade. Bộ chấm grade-lab-util có trong kho còn kiểm tra find và xargs, nhưng không bao gồm cp, tree, du, diff. Vì chưa chạy toàn bộ bộ chấm đó trong lần kiểm thử này, báo cáo không ghi điểm chấm MIT hoặc quy đổi 77/77 thành điểm trên 100.

6. Khó khăn gặp phải và giải pháp khắc phục

1. **Vấn đề 1: Tràn số lượng File Descriptor trong bài primes.**

   Nguyên nhân: Tiến trình con kế thừa toàn bộ bảng file descriptor của tiến trình cha nhưng không đóng kịp thời các đầu ghi của các stage trước đó, dẫn đến cạn kiệt tài nguyên (vượt quá NOFILE trong kernel).

   Giải pháp: Đóng ngay lập tức các đầu pipe không dùng ở cả cha và con trước khi bước vào vòng lặp xử lý dữ liệu.

2. **Vấn đề 2: Lặp vô hạn khi duyệt thư mục trong tree và du.**

   Nguyên nhân: Bỏ sót việc lọc hai liên kết đặc biệt là thư mục hiện hành (.) và thư mục cha (..).

   Giải pháp: Bổ sung kiểm tra strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0 trước khi gọi hàm đệ quy.

7. Kết luận và hướng phát triển

Qua bài thực hành Lab 01, nhóm đã hoàn thiện toàn diện các kỹ năng lập trình không gian người dùng trên hệ điều hành xv6:

- Nắm vững cơ chế vận hành của system call và sự chuyển giao giữa User Space và Kernel Space.
- Làm chủ kỹ thuật điều khiển luồng tiến trình và đồng bộ hóa thông qua các cơ chế IPC cơ bản nhưng hiệu quả.
- Hiểu rõ cấu trúc tổ chức và lưu trữ dữ liệu của hệ thống tệp tin xv6 ở mức độ nhị phân.

Tài liệu tham khảo

[1] Russ Cox, Frans Kaashoek, Robert Morris, *xv6: a simple, Unix-like teaching operating system*, bản RISC-V, ngày 31/08/2024. Tham khảo chương 1 về system call và pipe, chương 8 về hệ thống tệp. https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf.

[2] MIT, học phần 6.1810, *Lab: Xv6 and Unix utilities*, Fall 2024. Yêu cầu và hướng dẫn cho sleep, pingpong, primes. https://pdos.csail.mit.edu/6.1810/2024/labs/util.html.

[3] Tài liệu hướng dẫn thực hành Hệ điều hành Lab 01 do Giảng viên bộ môn biên soạn.

[4] MIT PDOS, *xv6-riscv*, kho mã nguồn chính thức. Tham khảo cách tổ chức tiến trình, pipe, hệ thống tệp và chương trình ls. Mã nguồn thực hiện bài được đối chiếu trực tiếp trong thư mục xv6-labs-2024/ của dự án. https://github.com/mit-pdos/xv6-riscv.

[5] Russ Cox, *Bell Labs and CSP Threads*. Tham khảo mô hình sàng số nguyên tố bằng các tiến trình nối với nhau qua pipe. https://swtch.com/~rsc/thread/.

[6] MIT, học phần 6.1810, *Tools*, Fall 2024. Hướng dẫn chuẩn bị công cụ RISC-V, QEMU và môi trường chạy xv6. https://pdos.csail.mit.edu/6.1810/2024/tools.html.

[7] GNU Project, *GNU Coreutils Manual*, các mục *cp invocation* và *du invocation*. Tham khảo chức năng và cách sử dụng các tiện ích sao chép, thống kê dung lượng trên Unix. https://www.gnu.org/software/coreutils/manual/coreutils.html.

[8] GNU Project, *Comparing and Merging Files*, mục *Invoking diff*. Tham khảo cách diễn giải kết quả so sánh và mã thoát của diff. https://www.gnu.org/software/diffutils/manual/html_node/Invoking-diff.html.

[9] Microsoft Learn, *How to install Linux on Windows with WSL*. Tham khảo yêu cầu hệ thống, cài bản phân phối Ubuntu và kiểm tra phiên bản WSL. https://learn.microsoft.com/en-us/windows/wsl/install.

[10] Homebrew, *Installation*. Tham khảo cách cài đặt và yêu cầu môi trường macOS. https://docs.brew.sh/Installation.

[11] Homebrew Formulae, *riscv64-elf-gcc* và *qemu*. Tham khảo tên gói, phụ thuộc Binutils và công cụ biên dịch RISC-V trên macOS. https://formulae.brew.sh/formula/riscv64-elf-gcc; https://formulae.brew.sh/formula/qemu.

Các tài liệu GNU được dùng để tham khảo hành vi của tiện ích Unix; các chương trình trong bài chỉ thực hiện phạm vi yêu cầu của Lab 01, không có đầy đủ tính năng của phiên bản GNU.